

Discussion Unit: DSE Analytics, Always-On SQL (AOS)

- Discussed briefly, enabled in DSE Studio Discussion Unit 6210/6211
- One (two ?) settings in dse.yaml, `alwayson_sql_options:`
`enabled: true`
- More settings, more function

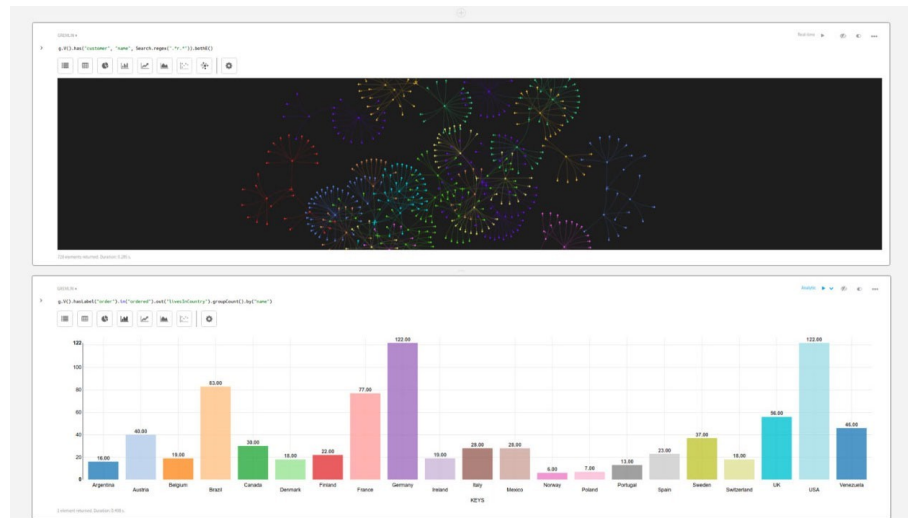
Discussion Lab:

Discuss-

DSE Analytics; Spark/SQL, SQL
(thus far)

DSE Analytics: Spark SQL

- Using "dse spark-sql"; How was your statement executed ? (Did you see an (n) step progress bar ?)
- What was your fastest return time ?
- What was happening that took a good portion of that time ?
- Was DSE Studio SQL faster ?





End of Discussion Lab:

DSE Always-On SQL (AOS):

```
dse.yaml,  
  alwayson_sql_options:  # Change T|F requires node restart  
    enabled: true
```

```
dse client-tool alwayson-sql  start|stop|status|restart|reconfig
```

Example,

```
dse client-tool alwayson-sql status // Also host:9077
```

```
dse client-tool alwayson-sql stop
```

```
dse client-tool alwayson-sql start
```

```
dse client-tool alwayson-sql --dc dc-west start
```

```
dse client-tool alwayson-sql reconfig
```

DSE Always-On SQL: reconfig



dse.yaml

- reserve_port_wait_time_ms
- alwayson_sql_status_check_wait_time_mas
- log_dsefs_dir
- runtime_max_errors

spark/conf/spark-alwayson-sql.conf
50 lines, mostly security

DSE Always-On SQL: What/Why-

Per analytics DC

Aimed at BI tools; Tableau, others

Offers a JDBC/ODBC like interface for that reason

And DSE Studio

Pre-instantiated resource, performance

DSE Always-On SQL: Has evolved with Spark



- > 2014
- Shark, SharkServer
 - Hive, Read HiveQL
 - Read Hadoop formats
 - M/R job atop Spark



- 2015
- Catalyst
 - Tree Manipulation FW
 - Spark streaming
 - Graphframes
 - Enable predicate pushdown
 - Support for TPC/DS
- Jobs on Spark FW (RDD, ..)

DSE Always-On SQL: Recent

DSE 5.1 used Spark ThriftServer (HiveServer2)

- Spark Thrift Server, not related to past DSE Thrift

DSE 6.0

- HA, Node in DC elected to be connection point (CP), failure, new election
- Driver transparently knows CP
- SQL syntax, cached to DSEFS (shared)
- Incremental collect()
- Uses native CQL comm protocol; async, and server notifications
- Must read Url on Notes page-

Thrift, Spark Thrift Server

- thrift.apache.org
- Fast/lightweight means to deliver RPC interface
- Security incompleteness; runs as single app, single user token (id)
- (No) H/A
 - No agent auto restart
 - Data not cached w/ new agent
- Single, long running app
 - No sharing w/ other apps, no shared SparkContext

DSE Always-On SQL: SQL commands

```
spark-sql> cache table ks_7579.customer;  
// Time taken: 10.445 seconds
```

```
spark-sql> cache table as select customer_num from ks_7579.customer;  
// Time taken: 1.239 seconds
```

```
spark-sql> uncache table ks_7579.customer;  
// Time taken: 0.158 seconds  
spark-sql> clear cache;  
// Time taken: 0.004 seconds
```

Sugar for df.cache
-- DSEFS or local disk as configured

DSE Always-On SQL: SQL commands

```
spark-sql> select count(*) from ks_7579.customer;  
// 28  
// Time taken: 9.78 seconds, Fetched 1 row(s)
```

```
spark-sql> insert into ks_7579.customer2 select * from ks_7579.customer;  
// Time taken: 2.934 seconds
```

```
spark-sql> select count(*) from ks_7579.customer2;  
// 28  
// Time taken: 0.546 seconds, Fetched 1 row(s)
```

DSE Always-On SQL: SQL commands, Explain

```
spark-sql> explain select count(*) from ks_7579.customer;
```

```
== Physical Plan ==
```

```
*HashAggregate(keys=[], functions=[count(1)])
```

```
+ - Exchange SinglePartition
```

```
  +- *HashAggregate(keys=[], functions=[partial_count(1)])
```

```
    +- *Scan org.apache.spark.sql.cassandra.CassandraSourceRelation  
       ks_7579.customer[] ReadSchema: struct<>
```

```
Time taken: 0.156 seconds, Fetched 1 row(s)
```



End of Unit: